

CREATE: A Closed-Loop Reward Editing Agent with Training Evidence for Reinforcement Learning

Anonymous Authors
Anonymous Submission

I. INTRODUCTION

Reinforcement learning (RL) acquires decision policies through interaction with an environment and has been applied to robotics, autonomous systems, games, and complex decision making. The reward function is the interface between task intent and policy optimization: it must express the desired behavior while providing informative and learnable feedback. A semantically plausible reward can nevertheless fail because of sparse signals, conflicting components, inappropriate scales, restrictive gates, or exploitable proxies, thereby degrading learning efficiency and final behavior quality [1]–[4].

Reward engineering is therefore an iterative process of task analysis, reward implementation, policy training, behavior inspection, and revision. A completed training run produces more than task return. Episode length, termination patterns, behavioral outcomes, component activation, component scale, and their evolution during training can reveal how a reward program shapes the learned policy. Converting these heterogeneous records into a precise reward modification, however, still requires substantial human analysis.

Code-capable large language models (LLMs) provide a new route to automating this process. Language to Rewards and Text2Reward translate task descriptions and environment interfaces into executable reward programs [5], [6]; Eureka combines reward-code generation with evolutionary optimization and reward reflection [7]; subsequent work incorporates dynamic feedback, structured reasoning, human feedback, and diagnostic refinement [8]–[11]. These studies establish that LLMs can generate and iteratively improve rewards, while also showing that reward quality becomes observable only after the resulting policy has been trained.

In parallel, language models are increasingly organized as agents that reason over observations, interact with external tools, and retain experience across trials. ReAct interleaves reasoning and environmental actions, whereas Reflexion and Self-Refine use feedback from previous attempts to improve later decisions [12]–[14]. These developments motivate an agent-oriented view of reward engineering in which policy training supplies observations and reward-code revision constitutes an external action.

This paper explores that view through CREATE, a *Closed-Loop Reward Engineering Agent with Training Evidence*. CREATE places a reward-editing agent above an inner RL policy learner. After each policy-training run, an auxiliary tool-using subagent retrieves and compresses native-task outcomes,

termination behavior, reward-component statistics, and training dynamics into a focused evidence summary. The reward-editing agent remains the sole decision maker: it identifies one principal reward semantic, selects a repair level, and applies a semantically localized reward edit. Intervention memory retains observations, hypotheses, edits, predictions, and outcomes, while a separate best archive protects the strongest executable reward.

The primary baseline and ablation study is conducted on LunarLander-v3. Budget-matched independent generation and stateless iterative revision test whether candidate count or repeated rewriting alone explains improvement, while two ablations isolate evidence enrichment and hierarchical semantic editing. BipedalWalker-v3 is used as a secondary cross-task validation of the complete CREATE workflow on continuous-action locomotion, without claiming a baseline comparison on that environment. Together, the experiments examine whether structured observation, persistent state, and bounded reward actions can convert failed policy training into reliable reward repair under a ten-evaluation budget.

II. RELATED WORK

A. Reward Engineering for Reinforcement Learning

Reward shaping augments a task objective with intermediate signals that improve credit assignment and exploration. Potential-based shaping identifies a class of transformations that preserves the optimal policy [2], while reward machines expose task structure through explicit automata [15]. Inverse reward design treats a specified reward as imperfect evidence of designer intent [3]. Related approaches infer objectives from demonstrations or preferences, or construct intrinsic rewards from novelty and prediction error. These methods reduce direct manual tuning, but typically require designer-specified features, demonstrations, preference data, or a predefined reward representation.

Reward functions have also been treated as objects of search and optimization. Evolutionary methods, meta-gradients, and behavior-based objectives adjust reward parameters or structures according to policy-training outcomes. This transition from manually fixed rewards to revisable reward programs provides the foundation for language-model-based reward engineering.

B. LLM-Based Reward Generation and Refinement

LLMs can map natural-language objectives and environment interfaces to executable reward code. Language to

Rewards generates dense reward programs for robotic skills, and Text2Reward produces interpretable free-form rewards for manipulation and locomotion [5], [6]. Eureka extends generation to evolutionary reward-code optimization and uses component feedback to guide subsequent candidates [7]. REvolve incorporates human feedback into reward evolution, while CARD combines reward generation with dynamic feedback and trajectory-level evaluation [8], [9].

Iterative refinement has also been organized through explicit reasoning. Self-Refine demonstrates that an LLM can improve an output through repeated feedback without parameter updates [14]. The CoT reward-engineering framework decomposes task understanding, reward-component construction, performance analysis, behavioral analysis, and improvement planning into a structured reasoning process [10]. Diagnostic-driven refinement further treats reward failure as a debugging problem and uses training diagnostics to guide targeted revision, while identifying calibration limits in continuous control [11]. Collectively, these methods show that training feedback can support reward revision beyond one-shot generation.

C. LLM Agents and Agentic Reward Engineering

Language-agent research studies how LLMs can maintain state and act in an external process. ReAct couples reasoning with tool-mediated actions, and Reflexion stores verbal feedback from prior trials as episodic experience [12], [13]. Tool use, specialized subagents, and intervention memory have subsequently enabled longer autonomous workflows.

Agentic mechanisms are also entering reward design. RF-Agent formulates reward construction as sequential decision making and uses Monte Carlo tree search to organize candidate generation and historical feedback [16]. STRIDE applies agentic engineering to automate reward design, deep-RL training, and feedback optimization for humanoid locomotion [17]. CREATE follows this agentic direction but centers the interaction interface on instrumented policy training. Its diagnostic subagent converts internal training records into observations, and the reward-editing agent commits to a semantically localized edit whose consequence is measured by the next complete training run.

III. CREATE

CREATE comprises an outer reward-editing agent and an underlying RL agent. The RL agent interacts with the environment and learns a policy under the current generated reward, whereas CREATE treats each completed policy-training and native-evaluation run as one reward-design round. As shown in Fig. 1, tool-assisted evidence processing organizes the resulting records before the reward-editing agent diagnoses the current reward and executes a validated edit.

The resulting observe–reflect–act loop is written as

$$\begin{aligned} O_t &= \Phi_{\mathcal{T}}(\mathcal{D}_t), \\ (H_t, A_t) &= \Psi(R_t, O_t, M_t, B_t), \\ R_{t+1} &= \mathcal{U}(R_t, A_t), \end{aligned} \quad (1)$$

where the training record $\mathcal{D}_t$ is retrieved and compressed into observation O_t through evidence tools. This processing stage organizes information but does not select a reward edit. The reward-editing agent combines O_t with the current reward R_t , intervention memory M_t , and best archive B_t to produce reflection H_t , reward action A_t , and the next executable reward program R_{t+1} .

A. Task Understanding and Reward Initialization

Given the task description, environment interface, and reward-code constraints, CREATE identifies the principal behavior semantics and instantiates them as named reward components. The initial reward is

$$R_0(s, a, s') = \sum_{i=1}^{m_0} w_i^{(0)} r_i^{(0)}(s, a, s'), \quad (2)$$

where each component represents a behavior semantic such as task progress, stability, target events, or action cost. Named components are logged independently during training, enabling component-level diagnosis in later rounds. Generated code is checked for interface and execution validity before policy training; prompts and validation rules are provided in the supplementary material.

B. Training-Evidence-Driven Observe–Reflect–Act

At iteration t , the underlying RL agent optimizes the generated reward, while CREATE evaluates the learned policy with the environment-native objective:

$$\begin{aligned} \pi_t &= \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{k=0}^{H-1} \gamma^k R_t(s_k, a_k, s_{k+1}) \right], \\ J_t &= \frac{1}{N} \sum_{n=1}^N \sum_{k=0}^{H_n-1} r_{\text{env}}(s_k^{(n)}, a_k^{(n)}, s_{k+1}^{(n)}). \end{aligned} \quad (3)$$

Thus, the generated reward shapes policy learning, whereas the native objective measures task completion. Each run produces episode outcomes, termination patterns, component activation and scale, temporal dynamics, and differences from the preceding reward version. An auxiliary tool-using subagent retrieves and compresses these records into a structured evidence summary. It reports what occurred during training but does not recommend coefficient changes, select an intervention level, or modify reward code.

Component statistics serve as diagnostic cues rather than fixed editing rules. Persistent inactivity may indicate an unreachable gate or sparse formulation; excessive activation and magnitude may reveal a dominant proxy; frequent activation with weak contribution may indicate insufficient scale, saturation, or an ineffective mathematical form. The reward-editing agent combines these cues with policy outcomes, reward source code, and prior interventions to identify one principal reward semantic for the next edit:

$$A_t = (q_t, \ell_t, \Delta_t), \quad \text{supp}(\Delta_t) \subseteq \mathcal{C}(q_t), \quad (4)$$

where q_t denotes the selected semantic, ℓ_t the edit level, and $\mathcal{C}(q_t)$ the components and parameters implementing that

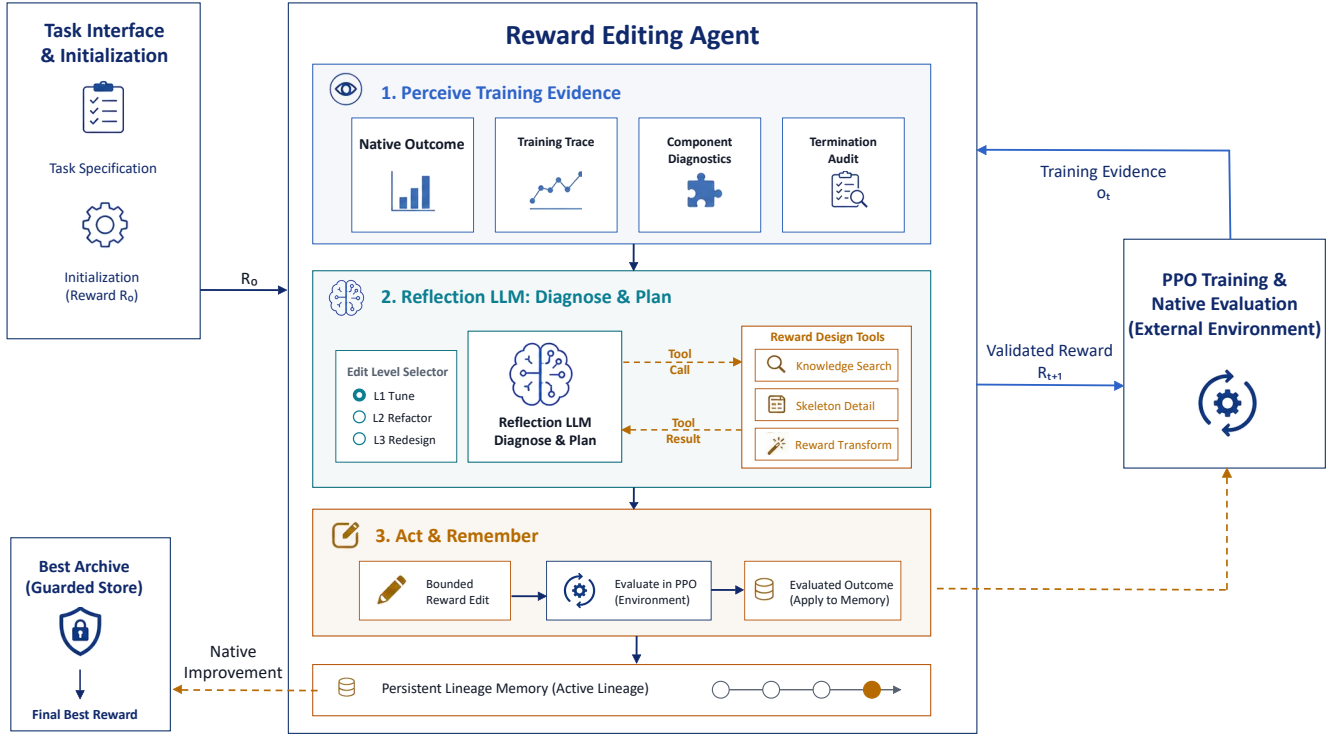


Fig. 1. Overview of CREATE. The underlying RL agent learns under the current generated reward. Tool-assisted evidence processing retrieves and compresses relevant training and lineage records, while the reward-editing agent remains responsible for failure diagnosis, semantic localization, intervention-level selection, and reward modification. Persistent memory records intervention outcomes, and the best archive retains the reward program with the highest native-task performance.

semantic. An ordinary action may add or remove a component, adjust its weight or gate, or replace its mathematical form. Related components may be adjusted jointly when they express the same semantic, but independent design intents are not mixed within one ordinary revision. L1 calibrates coefficients, thresholds, or gates; L2 adds, removes, or refactors components while preserving the target semantic; and L3 replans the reward structure after repeated local failure.

C. Intervention Memory and Best-Reward Selection

After each round, CREATE records the observation, diagnosed semantic, reward action, expected change, and subsequent outcome in persistent intervention memory. This history supports comparison across rounds and discourages repeated unsuccessful edits. The best archive is maintained separately from the active lineage to prevent later regressions from overwriting a previously discovered solution. After at most K reward evaluations, CREATE returns

$$t^* = \arg \max_{0 \leq t \leq K} J_t, \quad R^* = R_{t^*}. \quad (5)$$

The selected reward-policy pair is subsequently evaluated on independent episode seeds to assess stability.

D. Summary

Together, the three-stage architecture—evidence processing (Eq. 1), semantic diagnosis and bounded editing (Eq. 4), and intervention memory with best-reward selection (Eq. 5)—forms

a closed diagnostic loop. Each completed policy-training run produces component-level evidence that reveals *which* semantic failed and *how*; the editor commits to a targeted intervention on that semantic alone; and the next training run measures whether the diagnosis was correct. Intervention memory preserves this chain across rounds, while the best archive guards against regression. The experiments in Section IV and V test whether this diagnostic architecture can convert failed policy training into reliable reward repair under a fixed evaluation budget.

IV. EXPERIMENTAL PROCEDURE

This section details the experimental protocol that implements the CREATE framework described in Section III. We describe the setup, the six conditions under comparison, and the training and evaluation pipeline. Results are deferred to Section V.

A. Experimental Setup and Configuration

Experiments are conducted on two Gymnasium environments. LunarLander-v3 serves as the primary benchmark for baseline comparison and mechanism ablation; BipedalWalker-v3 provides a secondary validation on continuous-action locomotion. Every reward program trains a freshly initialized PPO policy through Stable-Baselines3. LunarLander uses 10^6 environment steps per candidate and BipedalWalker uses 5×10^6 , with four vectorized environments inside each policy-training job. The computational infrastructure comprises two NVIDIA GeForce

RTX 3090 GPUs (24 GB each), with DeepSeek V4 Pro serving as the LLM backbone for all LunarLander conditions. Full prompts, model settings, hyperparameters, and per-seed traces are provided in the supplementary material.

B. Condition Definitions

Six conditions are compared on LunarLander-v3 under a shared LLM backbone, environment description, reward interface, code checks, and PPO budget. Each condition contains five independent reward-search lineages and at most ten complete reward evaluations, so that every condition operates under the same total policy-training cost.

Single-shot generation evaluates only the initial reward produced by CREATE’s task-understanding phase (Section III-A), without subsequent observation or revision.

Independent-10 generates ten unrelated initial reward candidates, each from an independent invocation of the reward-initialization phase, and evaluates each with a separate policy-training run. The best candidate is selected post hoc. This condition spends the same maximum number of evaluations as CREATE but without cross-round information flow.

Stateless iterative revision repeatedly revises the current reward from basic current-round feedback (native fitness, episode length, termination statistics, component episode sums), but removes the evidence organizer, intervention memory, cross-round component comparison, and hierarchical semantic editing.

CREATE w/o evidence enrichment retains the full loop, memory, best archive, and hierarchical editing, but removes the tool-using evidence organizer: the editor receives native fitness, episode length, termination statistics, and component episode sums, but lacks activation rates, magnitude shares, temporal trends, and cross-round summaries.

CREATE w/o hierarchical editing retains enriched evidence, memory, and the best archive, but removes the semantic-localization constraint (Eq. 4), allowing unconstrained multi-component rewrites.

CREATE runs the complete architecture described in Section III.

C. Training and Evaluation Protocol

Each reward-design round consists of a complete PPO training run followed by native evaluation. The policy is optimized with the generated reward R_t , whereas candidate quality is measured only by the untouched environment reward r_{env} (Eq. 3). We define *search fitness* as the mean undiscounted native episodic return over 20 fixed evaluation episodes and *best search fitness* as the largest value retained within a lineage. After reward selection, the returned policy is evaluated over 100 unseen episodes; this is reported as *test fitness*. The task-solving thresholds are 200 for LunarLander and 300 for BipedalWalker.

Between rounds, the evidence organizer (Eq. 1, Φ_T) compresses training records into a structured summary. The reward-editing agent then selects one principal semantic, chooses an edit level (L1/L2/L3), and applies a bounded modification (Eq. 4). The modified program is validated before the next

TABLE I
LUNARLANDER-V3 REWARD SEARCH OVER FIVE LINEAGES. VALUES ARE MEAN $\pm$ SAMPLE STANDARD DEVIATION OF LINEAGE-WISE BEST SEARCH FITNESS.

Condition	Budget	Best fitness	Solved
Single-shot generation	1	-2.21 ± 73.38	0/5
Independent-10	10	-0.74 ± 114.40	0/5
Stateless revision	10	44.0 ± 65.9	0/5
CREATE w/o evidence enrichment	10	134.97 ± 148.43	2/5
CREATE w/o hierarchical editing	10	114.21 ± 46.31	0/5
CREATE	10	228.98 ± 16.54	5/5

round. The best archive is updated only when a new candidate exceeds the current incumbent in native search fitness (Eq. 5). On BipedalWalker-v3, only the complete CREATE workflow is evaluated; no baseline or ablation comparison is conducted on that task.

V. RESULTS AND ANALYSIS

A. Reward Search Under a Matched Training Budget

Search breadth and iterative rewriting alone do not produce reliable reward repair. Table I tests whether the diagnostic architecture of CREATE—rather than candidate count or repeated LLM invocation—accounts for performance. Single-shot generation never solves the task, demonstrating that an initially plausible reward is not reliably learnable: even semantically sound reward code can contain exploitable proxies or uninformative gradients that only emerge during policy training. Independent-10 fails across all five lineages despite evaluating the same maximum number of reward programs as CREATE, directly ruling out the alternative explanation that candidate count alone drives improvement. Ten independent guesses, each made without access to training dynamics, remain guesses: failed policy training contains diagnostic information that a fresh initialization cannot recover.

Iterative rewriting without persistent evidence produces non-monotonic progress. Stateless revision raises the mean lineage-wise best fitness to 44.0 but solves no seed. In three of five lineages, the initial reward is already the best version, and the final revision is worse than the stored best in all five. Each revision operates on the current reward text alone—without memory of which prior edits succeeded or failed, and without structured evidence to distinguish a poorly calibrated component from a fundamentally wrong semantic. The process therefore accumulates rewrites without accumulating progress. CREATE instead solves all five lineages at rounds 8, 10, 2, 4, and 3, requiring 5.4 complete reward evaluations on average to first cross the threshold. Its selected policies obtain 231.63 ± 12.23 test fitness, close to their search-time performance.

Fig. 2 reveals the qualitative difference in search dynamics. Independent generation improves only when another unrelated sample happens to be better—a random walk over reward space conditioned solely on initialization. Stateless revision reaches a low plateau despite repeated rewriting, confirming that revision without structured evidence cannot escape local basins. CREATE progressively raises its retained incumbent and eventually solves every lineage. The key distinction is not repeated LLM invocation, but whether a completed

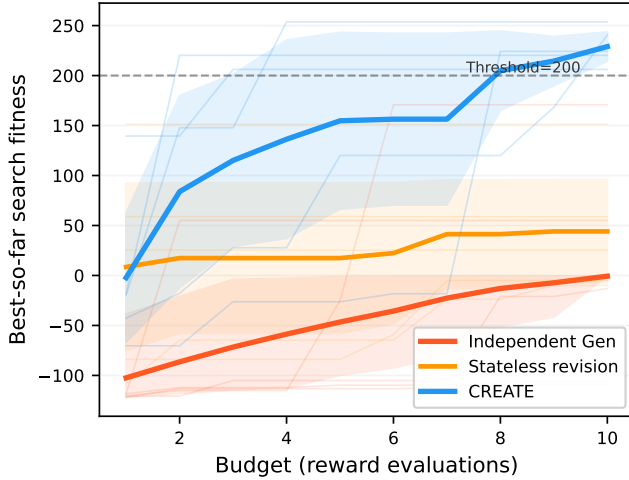


Fig. 2. Best-so-far LunarLander search fitness under a matched policy-training budget. Each curve averages five lineages after retaining the incumbent within each lineage; the dashed line denotes the task-solving threshold.

policy-training run becomes persistent diagnostic evidence: every CREATE edit is informed by component activation rates, magnitude shares, termination patterns, and cross-round trends, converting training failure from a binary outcome into actionable diagnostic information. Held-out test fitness (Fig. 3) confirms that the advantage persists under independent evaluation seeds: CREATE attains the highest mean with the lowest dispersion across all methods.

B. Mechanism Ablations

The previous section establishes that the closed loop works. We now isolate *why* it works by testing two mechanism hypotheses derived from the design choices in Section III.

H1 (Evidence enables diagnosis): the enriched evidence stream (activation rates, magnitude shares, temporal trends, cross-round summaries) enables the editor to distinguish between mechanistically distinct failure modes. Without this information, the editor operates on coarse aggregate statistics that conflate different root causes—an inactive component could be stuck behind an unreachable gate, too weak to compete, or behaviorally irrelevant, and a scalar episode sum cannot tell them apart.

Removing the tool-using evidence organizer leaves the editor with native fitness, episode length, termination statistics, and component episode sums, but removes activation rates, magnitude shares, temporal trends, and structured cross-round summaries. This variant solves two seeds but exhibits very high dispersion (134.97 ± 148.43 ; Table I), including one lineage that remains negative throughout all ten rounds. Basic component values can therefore support occasional repair, but cannot reliably answer the diagnostic questions that matter. H1 is supported: *what* training evidence is provided determines *which* diagnoses are possible.

H2 (Hierarchical editing enables single-trial attribution): the semantic-localization constraint (Eq. 4) ensures that each expensive retraining run serves as a controlled test

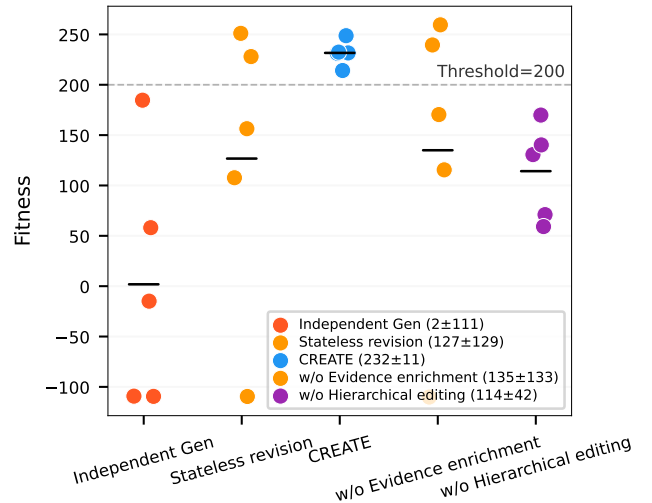


Fig. 3. Per-seed held-out test fitness and mechanism ablation results on LunarLander. Horizontal bars mark group means; the dashed line denotes the task-solving threshold (200).

of a single intervention hypothesis. Without this constraint, multi-component edits prevent the system from attributing a subsequent performance change to any specific modification—the credit assignment problem moves from RL training to reward design itself.

Removing hierarchical semantic editing retains enriched evidence, memory, and the best archive, but allows unconstrained multi-component rewrites. This variant reaches moderate fitness in several seeds yet solves none (114.21 ± 46.31), and one lineage collapses to approximately -5511 in two consecutive rounds. The failure is not a lack of information: the same enriched evidence is available, but when several unrelated components change together, one subsequent training result cannot identify which intervention should be retained or reverted. Each retraining run (10^6 environment steps) becomes an uncontrolled multivariate experiment rather than a focused hypothesis test. H2 is supported: bounded action is necessary for single-trial attribution.

Fig. 3 shows that H1 and H2 describe coupled requirements. Evidence-poor repair can occasionally succeed but is strongly initialization-dependent. Unconstrained editing has lower dispersion only because all five lineages remain below the threshold. Full CREATE combines a 5/5 solution rate with a substantially tighter distribution ($\sigma = 16.54$ vs. 148.43 and 46.31). Neither enriched observation nor bounded action alone suffices; together they enable the diagnostic closed loop.

C. How Evidence Becomes an Edit

Aggregate results and ablations establish *that* enriched evidence and hierarchical editing matter. We now examine *how* component evidence is translated into concrete edits by tracing three successful CREATE lineages on LunarLander. Fig. 4 shows the search fitness trajectory and edit-level labels for each; detailed activation-rate and magnitude-share heatmaps are provided in the supplementary material.

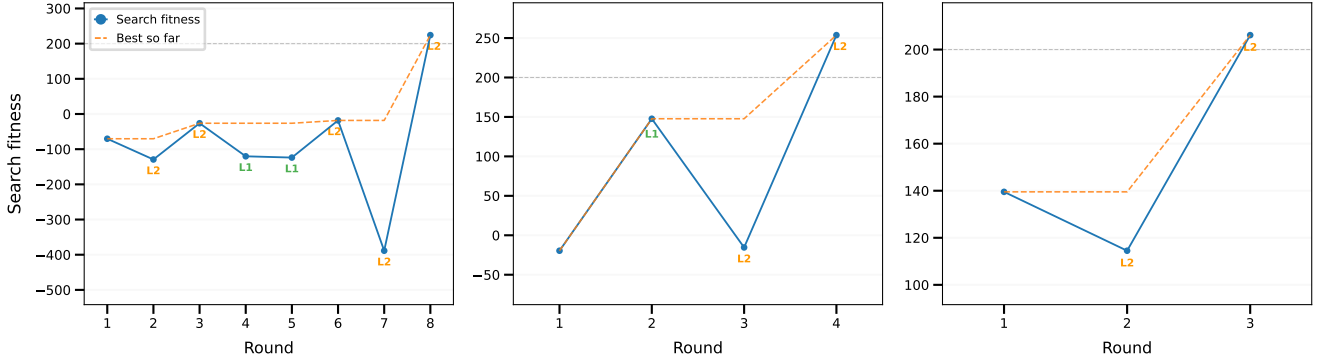


Fig. 4. Three representative CREATE repair trajectories on LunarLander: (a) collapse and recovery in seed 0, (b) progressive convergence in seed 3, and (c) proxy-misalignment repair in seed 4. Solid markers show search fitness per round; dashed line tracks the best-so-far incumbent. L1/L2 labels mark the edit level at each intervention.

Collapse and recovery (seed 0). Rounds 1–6 explore coefficient scaling, sparse-to-dense conversion, and mathematical-form changes without solving the task. Round 7 replaces a state-based proximity signal with differential progress shaping and collapses to approximately -389 . The evidence organizer reveals the mechanism: the new signal is near zero when the lander is stationary and becomes negative under disturbance, actively penalizing stability. The intervention memory retains the pre-collapse reward in the best archive. The editor uses this evidence to diagnose that the signal lacks a stable reference frame and applies an L2 intervention in round 8: restore a state-based formulation with an altitude factor that suppresses reward exploitation. Fitness rises to 224. The best archive guards this solution when the next exploratory reward falls to approximately -612 , demonstrating evidence-guided recovery and memory-gated regression prevention within a single lineage.

Progressive convergence (seed 3). Seed 3 improves from approximately -20 to 148 after an L2 refactor replaces an absolute-distance penalty with potential-difference approach shaping. The evidence organizer had reported that the absolute-distance term provided an uninformative gradient—constant penalty regardless of approach direction. An overly restrictive adjustment then causes a temporary regression. The intervention memory enables cross-round comparison: the pre-regression version achieved 148 with a specific contact-semantic formulation, localizing the regression to the most recent change. The subsequent L2 edit reconstructs the contact semantic as a reachable gradient from single-leg contact toward simultaneous stable contact; the next policy reaches 254. These revisions remain within a consistent signal framework, and the hierarchical editing constraint (H2) prevents simultaneous modification of the approach shaping and the contact semantic—an entanglement that would have made attribution impossible.

Proxy-misalignment repair (seed 4). Seed 4 begins at 140 but regresses after a local adjustment. The evidence organizer reports that a near-ground contact-transition reward is triggered approximately 75 times per episode during bouncing and dominates the shaped objective—over 60% of total shaped

TABLE II
BIPEDALWALKER-V3 RESULTS FOR COMPLETE CREATE ONLY. v DENOTES THE EVALUATED REWARD VERSION.

Seed	Initial fitness	First $v \geq 300$	Best fitness
0	270.7	v_2	320.0
1	280.5	v_2	313.7
2	272.1	v_2	307.9
3	290.0	v_2	311.1
4	103.0	v_3	304.9
Mean	243.26 ± 70.45	2.2 versions	311.54 ± 5.17

reward magnitude. The editor diagnoses a classic proxy-reward failure: the reward incentivizes repeated ground contact rather than controlled landing. The activation rate and magnitude share make this diagnosis possible because they reveal not *that* the reward is high but *which component* is inflated and *under what behavioral conditions*. The L2 repair replaces this transition proxy with persistent contact-quality gating: bounce velocity now suppresses the reward, and only low-speed contact remains favorable. The next version reaches 206 and solves the task. This trajectory demonstrates the full diagnostic chain: activation pattern $\rightarrow$ proxy identification $\rightarrow$ targeted semantic repair $\rightarrow$ verified improvement.

Across the three paths, the decisive edits address different defects—an uninformative gradient that became destructive, an unreachable contact gradient, and a rewardable proxy—but follow the same mechanism. Component activation and magnitude identify a principal semantic failure (H1); the editor commits to a targeted revision on that semantic alone (H2); and intervention memory preserves the best solution and enables cross-round comparison.

D. Cross-Task Validation

BipedalWalker-v3 checks whether CREATE remains operational on a continuous-action locomotion task. All five lineages cross the threshold by the third evaluated reward version (Table II); the selected policies obtain 310.82 ± 5.03 test fitness. Because no baselines or ablations were run on BipedalWalker, we do not claim comparative superiority on this environment, nor that the evidence-enrichment design—

developed and ablated on LunarLander—transfers without modification to locomotion domains.

Taken together, the experiments support a unified mechanistic account. CREATE converts each policy-training run from a binary evaluation into a diagnostic observation through structured evidence processing. Enriched evidence enables the editor to distinguish inactive, saturated, and proxy-dominated components (H1). Hierarchical editing ensures that each retraining run serves as a controlled hypothesis test whose outcome can be attributed to a single intervention (H2). Intervention memory preserves the evidence–action–outcome chain across rounds and prevents regression. These three components are coupled requirements: evidence without bounded editing provides diagnosis without attribution; bounded editing without evidence constrains action without guiding it; and memory amplifies both by enabling cross-round comparison. Failed policy training—when instrumented as structured evidence and paired with bounded diagnostic actions—can become a reliable signal for reward repair under a fixed evaluation budget.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proceedings of the 16th International Conference on Machine Learning*, 1999, pp. 278–287.
- [3] D. Hadfield-Menell, S. Milli, P. Abbeel, S. Russell, and A. Dragan, “Inverse reward design,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [5] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, “Language to rewards for robotic skill synthesis,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 374–404.
- [6] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, “Text2reward: Reward shaping with language models for reinforcement learning,” in *International Conference on Learning Representations*, 2024.
- [7] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, “Eureka: Human-level reward design via coding large language models,” in *International Conference on Learning Representations*, 2024.
- [8] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, and X. Li, “A large language model-driven reward design framework via dynamic feedback for reinforcement learning,” *Knowledge-Based Systems*, vol. 326, p. 114065, 2025.
- [9] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires, “REvolve: Reward evolution with large language models using human feedback,” in *International Conference on Learning Representations*, 2025.
- [10] X. Zhu, J. Du, Q. Fu, and L. Chen, “LLM-based reward engineering for reinforcement learning: A chain of thought approach,” in *2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, 2025.
- [11] Y. Wang, Y. Tang, B. Liu, X. Liu, and D. Shang, “When LLM reward design fails: Diagnostic-driven refinement for sparse structured RL,” *arXiv preprint arXiv:2605.28918*, 2026.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [15] R. T. Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, “Reward machines: Exploiting reward function structure in reinforcement learning,” *Journal of Artificial Intelligence Research*, vol. 73, pp. 173–208, 2022.
- [16] N. Gao, X. Zhang, X. Jiang, M. You, M. Zhang, and Y. Deng, “RF-Agent: Automated reward function design via language agent tree search,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [17] Z. Wu, J. Lu, Y. Chen, Y. Liu, Y. Zhuang, and L. Hu, “STRIDE: Automating reward design, deep reinforcement learning training and feedback optimization in humanoid robotics locomotion,” *arXiv preprint arXiv:2502.04692*, 2025.